

Lecture 10: Array Problem Solving (Two Pointers, XOR Logic, & Sorting)

1. Core Algorithmic Paradigms

This module transitions from basic array traversal to optimal mathematical and structural manipulation using common competitive programming paradigms.

- **The XOR Trick (^):** The bitwise XOR operator has two critical mathematical properties used for eliminating duplicates:
 - **Self-Cancellation:** $X \oplus X = 0$. Any number XORed with itself perfectly cancels out to zero.
 - **Identity Property:** $X \oplus 0 = X$. Any number XORed with zero remains unchanged.
- **Two Pointers:** Instead of using nested `for` loops ($O(N^2)$), we initialize two index trackers (pointers) that traverse the array simultaneously from different directions, reducing time complexity to $O(N)$.

Problem-Solving Deconstruction

Problem 1: Swap Alternate Elements

1. **Problem Statement & Constraints:** Given an array, iterate through and swap consecutive adjacent pairs of elements.
2. **Core Intuition:** We step through the array jumping by 2 indices at a time. To prevent segmentation faults (accessing memory outside the array), we strictly verify that the adjacent element (`i+1`) exists within the array boundary before attempting a swap.

3. Algorithmic Steps:

1. Open a `for` loop initializing iterator `i = 0`.
2. Set the update condition to jump in pairs: `i += 2`.
3. Check boundary condition: `if(i + 1 < size)`.
4. If True, execute the built-in swap: `swap(arr[i], arr[i+1])`.
5. Continue until the loop terminates.

4. Dry Run:

Input: `[1, 2, 3, 4, 5]`

1. `i = 0`. Check `0 + 1 < 5` (True). Swap `arr[0]` and `arr[1]`. Array: `[2, 1, 3, 4, 5]`.

- `i = 2` . Check `2 + 1 < 5` (True). Swap `arr[2]` and `arr[3]` . Array: `[2, 1, 4, 3, 5]` .
- `i = 4` . Check `4 + 1 < 5` (False). Loop terminates.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The array is strictly traversed once, jumping by 2.
Space Complexity	$O(1)$	Swapping executes entirely in-place with no auxiliary arrays.

Problem 2: Find Unique Element (CodeStudio)

- Problem Statement & Constraints:** Given an array of size $N = 2M + 1$ where exactly M numbers are present twice and one number is present exactly once, find the unique number.
- Core Intuition:** Utilizing the XOR properties ($A \oplus A = 0$ and $B \oplus 0 = B$). By chaining an XOR operation across the entire array, all duplicated pairs will mathematically cancel themselves out to 0, leaving only the unique element isolated in the accumulator.

3. Algorithmic Steps:

- Initialize a tracker variable `ans = 0` .
- Iterate `i` from 0 to `size - 1` .
- Cumulatively XOR the current array element into the tracker: `ans ^= arr[i]` .
- Return `ans` .

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	A single, linear pass over the elements.
Space Complexity	$O(1)$	Strictly constant space overhead (<code>ans</code>) .

C++

```
int findUnique(int *arr, int size)
{
    int ans = 0;
    for (int i=0; i<size; i++) {
        ans ^= arr[i]; // Duplicates self-cancel to 0
    }
}
```

```
    return ans;
}
```

Problem 3: Find Duplicate in Array (CodeStudio)

1. Problem Statement & Constraints: Given an array of size N containing elements uniquely spanning from 1 to $N-1$, there is exactly one element that appears twice. Find it.

2. Core Intuition: This involves a two-stage XOR cancellation. First, we XOR all elements physically inside the array together. Second, we XOR that result with a perfectly continuous sequence of integers from 1 to $N-1$. Every single number will now have appeared twice (and canceled to 0) except for the duplicated number, which will have appeared three times. XORing a number three times yields the number itself ($X \oplus X \oplus X = X$).

3. Algorithmic Steps:

1. Initialize `ans = 0`.
2. Traverse the input array and XOR all values: `ans ^= arr[i]`.
3. Traverse a simulated range from 1 to `size - 1` and XOR against the tracker: `ans ^= i`.
4. Return `ans`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	Executes two non-nested linear loops.
Space Complexity	$O(1)$	State maintained inside a single <code>ans</code> integer.

C++

```
int findDuplicate(vector<int> &arr)
{
    int ans = 0;
    // XOR all elements in the physical array
    for (int i=0; i<arr.size(); i++) {
        ans ^= arr[i];
    }
    // XOR all indices from 1 to N-1
    for (int i=1; i<arr.size(); i++) {
        ans ^= i;
    }
    return ans;
}
```

Problem 4: Intersection of Two Sorted Arrays

1. Problem Statement & Constraints: Given two sorted arrays `arr1` (size N) and `arr2` (size M), return a new array containing their exact intersection (common elements).

2. Core Intuition: Because both arrays are strictly sorted, we do not need $O(N^2)$ nested loops to search. We can deploy the **Two Pointers Approach**. We place a pointer at the beginning of each array. We compare the pointer values and strictly advance the pointer pointing to the smaller value to "catch up" to the larger value.

3. Algorithmic Steps:

1. Initialize `i = 0` (for `arr1`), `j = 0` (for `arr2`), and an empty `ans` vector.
2. Open a `while` loop bound by `i < n && j < m` (stops if either array is exhausted).
3. **Match Found:** If `arr1[i] == arr2[j]`, push the element to `ans`, and advance both pointers (`i++`, `j++`).
4. **Pointer 1 lags:** If `arr1[i] < arr2[j]`, advance `i++` to find larger values.
5. **Pointer 2 lags:** If `arr1[i] > arr2[j]`, advance `j++`.
6. Return the `ans` vector.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\max(N, M))$	Pointers traverse both arrays at most once sequentially.
Space Complexity	$O(\min(N, M))$	Worst case, all elements of the smaller array match and are stored in the answer vector.

C++

```
#include <bits/stdc++.h>
vector<int> findArrayIntersection(vector<int> &arr1, int n, vector<int> &arr2,
int m)
{
    int i=0, j=0;
    vector<int> arr;

    while (i<n && j<m) {
        if (arr1[i]==arr2[j]) {
            arr.push_back(arr1[i]);
            i++;
            j++;
        }
    }
}
```

```

    }
    else if (arr1[i]<arr2[j]) {
        i++;
    }
    else {
        j++;
    }
}
return arr;
}

```

Problem 5: Sort 0 1 2 (Dutch National Flag Algorithm)

1. Problem Statement & Constraints: Given an array strictly consisting of only 0s, 1s, and 2s, mathematically sort the array in ascending order entirely in-place without utilizing standard sorting algorithms.

2. Core Intuition: The array can be segmented into three distinct tracking sections using three pointers: `low`, `mid`, and `high`.

- **0s** map to the left boundary (`low`).
- **2s** map to the right boundary (`high`).
- **1s** logically fall into place in the middle as `mid` evaluates them.

3. Algorithmic Steps:

1. Initialize pointers: `low = 0`, `mid = 0`, `high = n - 1`.
2. Open a `while` loop bound by `mid <= high`.
3. Evaluate `arr[mid]`:
 - **Case 0:** Swap `arr[mid]` and `arr[low]`. The zero is correctly placed. Advance `low++` and `mid++`.
 - **Case 1:** The element is already in the correct relative zone. Simply advance the evaluation pointer `mid++`.
 - **Case 2:** Swap `arr[mid]` and `arr[high]`. The two is pushed to the back. Retract `high--`. (Critically, do NOT advance `mid` here, because the element swapped back from `high` has not been evaluated yet).

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The array is processed exactly once via the sliding <code>mid</code> and <code>high</code> pointers colliding.
Space Complexity	$O(1)$	Strictly manipulated in-place using scalar pointers.

```

#include <bits/stdc++.h>
void sort012(int *arr, int n) {
    int low = 0;
    int mid = 0;
    int high = n-1;

    while (mid <= high) {
        if (arr[mid] == 0) {
            swap(arr[mid], arr[low]);
            low++;
            mid++;
        }
        else if (arr[mid] == 1) {
            mid++;
        }
        else {
            swap(arr[mid], arr[high]);
            high--;
        }
    }
}

```

Problem 6: 3Sum (Find Triplets Summing to K)

1. Problem Statement & Constraints: Find all unique mathematical triplets inside an array that sum to a target integer K .

2. Core Intuition: A brute force solution requires three nested loops ($O(N^3)$). By sorting the array first, we can lock in one value with an outer loop (i), and treat the remainder of the array as a "Two Pointer Pair Sum" problem ($left$ and $right$). We utilize a C++ `set` to automatically filter out duplicate triplet vectors.

3. Algorithmic Steps:

1. Sort the input array `arr`.
2. Iterate `i` from 0 up to $N-2$ (leaving room for 2 more pointers).
3. Initialize pointers: `left = i + 1` and `right = n - 1`.
4. Run a `while (left < right)` loop calculating `sum = arr[i] + arr[left] + arr[right]`.
5. **Logic Routing:**
 - If `sum == K`, store the ordered triplet in the `set`, advance `left++` and retract `right--`.
 - If `sum > K`, we are over-target. Shrink the highest number by retracting `right--`.
 - If `sum < K`, we are under-target. Grow the lowest number by advancing `left++`.

6. Translate the `set` into a 2D `vector` and return.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N^2 \log(\text{unique_triplets}))$	The $O(N^2)$ scaling comes from the nested traversal. The <code>set</code> insertion inherently adds logarithmic overhead.
Space Complexity	$O(\text{Number of Unique Triplets})$	Space is dominated by the auxiliary storage required to maintain the Set and Vector response matrices.